

Notes on Instant-NGP to PBRT-v4 integration

Adding Testbed to WavefrontPathIntegrator

There are potentially many different ways I could go about integrating the Instant-NGP network, as all I really need are the network weights/checkpoint and I could create a custom class within PBRT to load it up and initialize some model using `tiny-cuda-nn` as well as utilize some of the other classes/structures that PBRT uses to deal with GPU side memory.

However, I decided to just include `Testbed` as a member variable within the `WavefrontPathIntegrator` instead. There are a couple reasons for this

1. Saves me a lot of time from having to create initialization and clean up code for `tiny-cuda-nn` network and I need to balance how much time I can allocate to this part
2. `WavefrontPathIntegrator` is the only integrator in this project that I am replacing a GPU kernel for, so the neural network code is not needed anywhere else
3. Most of the functionality I need is already implemented in `Testbed` and is robust enough to handle or log errors

The one downside is because I am just linking against `ngp` in the `CMakeLists.txt` file, the build times will get quite a bit longer as it needs to also build `ngp` as well. In terms of how the network itself is integrated into the wavefront architecture, it requires some thought.

The way that data is fed into the `SampleMediumInteraction` kernel is via `MediumSampleWorkItems` in a `MediumSampleQueue` all wrapped in a `ForAllQueued` wrapper. However, the contents of the queue or not in the shape that is required for the `ngp` network, as it contains other extraneous things that my network does not actually handle

```
// MediumSampleWorkItem Definition
struct MediumSampleWorkItem
{
    // Both enqueue types (have mtl and no hit)
    Ray ray;
    int depth;
    Float tMax;
    SampledWavelengths lambda;
    SampledSpectrum beta;
    SampledSpectrum r_u;
    SampledSpectrum r_l;
    int pixelIndex;
    LightSampleContext prevIntrCtx;
    int specularBounce;
    int anyNonSpecularBounces;
    Float etaScale;

    // Have a hit material as well
    Light areaLight;
    Point3fi pi;
    Normal3f n;
    Vector3f dpdu, dpdv;
    Vector3f wo;
    Point2f uv;
    Material material;
    Normal3f ns;
    Vector3f dpdus, dpdvs;
    Normal3f dndus, dndvs;
    int faceIndex;
    MediumInterface mediumInterface;
};
```

Listing 1: Structure of Medium Sample Work Item

Thus, I can't just feed these directly into the kernel. I need to first prepare the data by normalizing and writing into appropriate buffers, which would then be sent for inference and then read back and operated on to create new ray samples or for the next stage. Thus, overall, the stages look like this:

1. Run a `ForAllQueued` to prepare and normalize a batch of ray origins, directions and `tMax` values into buffers
2. Send these buffers into pinned memory or to the GPU for the Instant-NGP network
3. Perform inference via the network which are written to an output buffer
4. Read the network outputs and use them to calculate the added radiance and new throughput values for the rays

5. Enqueue work items for the next kernel in the wavefront architecture depending on if the ray escaped or hit a surface (So either `EscapedRayWorkItem`, `MaterialEvalWorkItem`, `HitAreaLightWorkItem`)

The important part to consider is the fact that because the network infers its calculation based on the ray fully going through the medium without scattering and just adds on its contribution as if it did scatter through the medium, we do NOT enqueue any medium scattering work items. This is because the network replaces the entire delta tracking loop as well (e.g. in the original `SampleMediumInteraction` kernel, if the ray scattered it would also enqueue a `SampleMediumScattering` work item which would also shoot out shadow rays and calculate some indirect lighting as well, but the model does consider this).

This does, then, lead to a good thing to consider for discussion or for improvements for the future. This can do medium interaction but doesn't replace or even call the `MediumScattering` kernel and thus in the future, the model could also be used to predict whether a ray sample should also scatter or not which would then be used in the `SampleMediumInteraction` kernel to decide to enqueue a `MediumScatterWorkItem` as well

Adding a “Slot Map” for beta tracking

I do a Pre and Post `ForAllQueued`, the Pre in order to perform data packing and normalization into the inference loop. However, work items are executed in a nondeterministic order, the first work item may not be operated on first. Furthermore, I need to allocate buffers to pass data into the network and a buffer to get data out of the network. But I can't use the `pixelIndex` since it operates in scanlines of the image. I could allocate a massive buffer for the number of pixels but this seems wasteful since every loop will only use a fraction of it. Furthermore, the output buffer for the network is not the same as the max pixel index as the buffer can only be in multiples of the batch size. I also need to ensure in the second loop that the corresponding rays get the corresponding values in the output buffer. Therefore, in the pre loop, it decides on what “slot” it has in the buffer via an `atomicAdd`, so work item 1 might get slot 1 but work item 2 might get slot 6 depending on execution, its not deterministic.

Thus in the Post loop, which is done in order to actually launch new work items for other kernels, to access buffers needed (such as `d_betaAfter`) it looks up its slot into the buffer via its `pixelIndex` (which is set when claiming the slot in the pre loop) and uses this to index into `d_betaAfter` and other related carry through buffers. In other words, `d_slotMap[w.pixelIndex] → slot` and pre loop writes the mapping and post loop uses it.

Dealing with `r_u` and `r_l`

Firstly, `r_u` and `r_l` are additional delta tracking variables used in PBRT for Multiple Importance Sampling (MIS). These are weights PBRT uses for robust ratio tracking in heterogeneous media, so in homogeneous media these are less important. In the original delta-tracking, every time beta is scaled, `r_u` and `r_l` is also scaled by a similar amount.

The downstream MIS computation depends on the ratio between beta, `r_u` and `r_l` being consistent so beta can't just be passed through without being updated. Luckily since the output throughput value is a similar value used to update them, they can be updated similarly, but just needs pass through buffers similar for the beta, so `d_ruBefore`, `d_ruAfter`, `d_rlBefore`, `d_rlAfter` buffer. They may not be completely correct as the network isn't optimized for these but since its just about an approximation, this was the best solution I can think of.

Dealing with the next kernels

For dealing with the next kernels, I mainly copy what the original `SampleMediumInteraction` function had but with some minor caveats. The work items that I consider generating/kernels I consider are:

- `EscapedRayQueue(EscapedRayWorkItem)`
- `HitAreaLightRayQueue(HitAreaLightWorkItem)`
- `MaterialEvalQueue(MaterialEvalWorkItem)` (well, the corresponding surface material's queue)

One major queue/kernel that I do not enqueue that the original does is the `MediumScatterQueue` that deals with medium scattering for the reasons given above